第 3-3 讲 • 读第二遍：类型与契约

> **模块 3 • 学会读代码** ｜ 建议用时 **40 分钟**（阅读 20 + 动手 20）
> 前置要求：完成 3-1、3-2（会追数据流）

本节目标

读完并做完这一讲，你能：

1. 说清**类型（Type）**是什么，以及它在**写代码时**（不是运行时）帮你做什么。
2. 读懂 `shared\types.ts` —— 这个 **505 行、53 个类型**的文件为什么是整个项目的"字典"。
3. 理解**契约（Contract）**的含义：为什么改一个字段要动好几个文件。
4. 掌握一个实用技能：**用类型反推业务**——看到一个类型定义，就知道这块业务需要什么数据。
5. 亲手做一次"影响面分析"：假设给记录加一个字段，找出**所有要改的地方**。

一、类型是什么

1.1 现实比喻：快递单的格式要求

寄快递时，单子上必须填：**寄件人、收件人、电话、地址**。

- **格式清楚**：填单的人知道该写什么，快递员知道该读什么。
- **格式不清**：有人只写"寄给老王"，包裹就送不出去。

类型就是这个"格式要求"，只不过它约束的是数据：

```
interface RecordInput {
  date: string          // 必须是一段文字（日期）
  amount: number        // 必须是一个数字（金额，单位：分）
  type: RecordType      // 必须是 income / expense / transfer 之一
  note: string          // 必须是一段文字
  tags: string[]        // 必须是一个字符串数组
}
```

这段代码在说：

> "谁要给我一笔账，必须按这个格式来：日期是文字、金额是数字、类型是三种之一、标签是一个列表。"

1.2 类型在什么时候起作用

这是关键：类型**只在写代码时起作用，运行时完全不存在。**

你在第 1-3 讲已经实测验证过这一点：

```
源码里的 ': string' 出现次数：31 次
产物里的 ': string' 出现次数：0 次      ← 构建时全被删掉了
```

所以：

阶段	类型的作用
你写代码时	编辑器立刻标红："`date` 你传了个数字，应该传文字"
构建时	类型标注被**删除**（只留 JavaScript）
运行时	类型**不存在**。程序不会因为你传错类型而报错——它只会算出错误结果

这意味着两件事：

1. **类型是"开发时的护栏"**，不是"运行时的保险"。它防的是**你写错**，不是**用户乱输**。
2. **用户输入必须另外校验**——这就是 3-2 讲里 `validate()` 函数存在的原因。**类型管开发者，校验管用户。**

这个区分很重要：很多初学者以为"我定义了类型就很安全了"，其实**类型和校验是两个不同的防线，都必须有。**

二、`shared\types.ts`：项目的字典

2.1 它有多大

项目	数值
行数	**505 行**
导出的类型/接口	**53 个**

为什么一个文件能装下所有类型：因为类型只是"描述形状"，不包含任何逻辑。**53 个形状，说明了这个软件能处理 53 种数据。**

2.2 为什么放在 `shared\` 而不是各放各的

回顾 2-1 讲的架构：主进程和渲染进程是**两个独立的进程**，各自有各自的内存。

但它们必须对数据有完全一致的理解：

- 界面说："我传给你一个 `payload`，里面有 `amount`。"
- 主进程必须知道："`amount` 是什么类型？单位是元还是分？"

如果两边各写一份定义，早晚会不一致：

```
界面里写：amount: number    （以为是元）
主进程以为：amount 是分
→ 用户记 12 元，存成了 12 分
```

放在 `shared\` 就杜绝了这种可能：**只有一份定义，两边都引用它。**

这就是"契约"的第一层含义：**双方共同承认的一份约定。**

2.3 契约的第二层：`Api` 接口

`src\shared\ipc.ts` 里有一个特别重要的类型：

```
export interface Api {
  app: {
    bootstrap: () => Promise<BootstrapInfo>
    initialize: (payload: {...}) => Promise<BootstrapInfo>
    ...
  }
  records: {
    list: (query: RecordQuery) => Promise<RecordListResult>
    create: (input: RecordInput) => Promise<LedgerRecord>
    ...
  }
  ...
}
```

这个 `Api` 类型是"界面能调什么"的完整清单（和 2-2 讲的 preload 白名单是同一件事的两个视角）：

视角	文件	回答的问题
实现	`preload\index.ts`	我真的暴露了哪些方法
契约	`shared\ipc.ts` 的 `Api`	我**承诺**暴露哪些方法

两者必须一致。如果不一致，TypeScript 会直接报错：

```
const api: Api = { ... }   // 少一个方法 → 立刻报错
```

这就是类型的威力：它把"口头承诺"变成了**编译时检查的硬约束。**

三、为什么改一个字段要动好几个文件

这是你以后最常遇到的困惑。用**具体例子**讲清楚。

3.1 场景：给记录加一个"商家"字段

假设产品需求是：**每笔账可以记录商家名称**（比如"水果店""超市"）。

看起来只是个输入框而已，但实际要改的地方：

#	要改的地方	为什么
1	`shared\types.ts` 的 `RecordInput`	加 `merchant: string` —— **先改契约**
2	`shared\types.ts` 的 `LedgerRecord`	读出来的数据也要有这个字段
3	`db\migrations\004_add_merchant.ts`	**新建迁移**，给表加列（不能改 001，见 2-3 讲）
4	`db\repositories\records.repo.ts`	`INSERT` 语句要加上这一列
5	`db\repositories\records.repo.ts`	`SELECT` 语句要取出这一列
6	`main\ipc\records.ts` 的 `validate()`	校验新字段（长度限制等）
7	界面 `RecordForm.tsx`	加输入框，并放进 `payload`
8	界面列表组件	决定要不要显示它

8 个地方。 而 TypeScript 会帮你**自动找出其中大部分。**

我实测验证过这件事：我临时给 `RecordInput` 加了一个字段 `merchantTEST`，然后跑类型检查——

步骤	结果
加字段前	`npm run typecheck` 报错 **0 条**
加了 `merchantTEST` 后	报错 **24 条**，分布在这些文件里：
	`demo.ts`（4 处）、`exchange-probe.ts`（3 处）、`persist-probe.ts` ……
把字段删掉后	报错回到 **0 条**

这 24 条报错就是那份"自动生成的待办清单"。 每一条都在说："这里也要处理新字段。"

这里有个特别值得注意的发现：这 24 条报错**几乎全都在测试和示例数据文件里**，而不是在业务代码里。

为什么：因为业务代码（`repo.create`、`RecordForm`）用的是**同一个契约对象**，加字段后它们**可能不需要改**（比如 `INSERT` 语句只取它认识的列）。而**造测试数据的地方必须列出每一个字段**，所以它们立刻报错。

这告诉我们一件事：**类型检查的报错位置，能反映出"哪些地方是硬编码的、哪些地方是跟着契约走的"。**硬编码越多的地方，就是改动成本越高的地方。

3.2 一个真实的数据：`tags` 字段出现在 14 个文件里

```
(Get-ChildItem src -Recurse -Include *.ts,*.tsx | Select-String "\btags\b" | Group-Object
```

输出：14。

也就是说，"标签"这一个功能，牵涉 14 个文件：类型定义、数据库读写、校验、界面表单、列表显示、CSV 导入导出、截图导入……

这解释了两件事：

1. **为什么软件越大改起来越慢**：不是代码写得烂，而是**功能之间真的有关联。**
2. **为什么要有类型系统**：14 个文件里的字段名如果写错一个（`tag` vs `tags`），**类型检查会立刻发现。**

3.3 改动的正确顺序

记住这个顺序（模块 4 会反复用到）：

```
① 先改契约（types.ts）
      ↓ 类型检查会报错，告诉你哪里还没改
② 改数据层（迁移 + 仓储层）
      ↓
③ 改接口层（校验）
      ↓
④ 改界面
      ↓
⑤ 跑测试（模块 5）
```

为什么"先改契约"：因为契约是**中心。**改中心，所有引用它的地方都会亮红灯——**这就是一份自动生成的"待办清单"。**

反过来做（先改界面）会怎样：你改了界面，但契约没变，于是**类型检查可能还是通过的**（因为界面上多传一个字段不报错）。**你就失去了那份自动待办清单，得靠自己记住要改哪儿——必然漏。**

四、用类型反推业务

这是一个很实用的读代码技巧：`types.ts` 里 53 个类型，每一个都在告诉你"这块业务需要什么数据"。

4.1 例子：从 `Account` 看账户业务

```
export interface Account {
  id: string
  name: string
  icon: string
  color: string
  initialBalance: number      // 初始余额
  sortOrder: number           // 排序
  createdAt: number
  updatedAt: number
  deletedAt: number | null    // 软删除（2-3 讲）
  archivedAt?: number | null  // 归档
}
```

从这 10 个字段能读出什么业务：

字段	说明的业务
`initialBalance`	账户要有"期初余额"（不是从 0 开始算）
`sortOrder`	账户**可以拖动排序**
`color` + `icon`	账户**可以自定义外观**
`deletedAt`	删除是软删除
`archivedAt`	**有"归档"功能**（不用了但不删）

一行代码没看，只靠类型定义，你就知道了账户功能的全部要点。 这就是"用类型反推业务"。

4.2 例子：从 `RecordQuery` 看筛选能力

```
export interface RecordQuery {
  from?: string
  to?: string
  keyword?: string
  categoryIds?: string[]
  accountIds?: string[]
  types?: RecordType[]
  channels?: string[]
  amountBrackets?: AmountBracket[]
  includeDeleted?: boolean
  onlyDeleted?: boolean
  limit?: number
  offset?: number
}
```

读出来的业务：

- 支持**日期区间**筛选（`from` / `to`）
- 支持**关键词**搜索（`keyword`）
- 支持**多分类、多账户、多类型、多渠道**筛选（都是数组）
- 支持**金额档位**筛选（`amountBrackets`，说明界面上有"0-50 / 50-200"这种档位选择）
- 支持**回收站**（`onlyDeleted`）
- 支持**分页**（`limit` / `offset`）

注意 `?` 的含义：`from?: string` 里的问号表示"**这个字段可以不传**"。所以这些筛选条件**都是可选的**——用户可以只用其中一部分。

这就是类型的表达力：`?` 一个字符，说明了"这个条件是可选的"。

五、动手任务

步骤 1：量化这个"字典"的规模

```
cd "C:\Users\34123\Desktop\记账簿"
(Get-Content src\shared\types.ts | Measure-Object -Line).Lines
(Select-String -Path src\shared\types.ts -Pattern "^export (interface|type)").Count
```

预期：**505 行、53 个类型。**

再看一眼类型名清单：

```
Select-String -Path src\shared\types.ts -Pattern "^export (interface|type) (\w+)" | ForEac
```

扫一遍这些名字，你会发现**每一个都对应一块业务**：`Account`（账户）、`Category`（分类）、`Budget`（预算）、`StatsSummary`（统计）……

这就是整个软件的业务地图。

步骤 2：读一个类型，反推业务

挑一个你没见过的类型，读出它的业务含义：

```
Select-String -Path src\shared\types.ts -Pattern "export interface BudgetWithProgress" -Co
```

自己试着回答：

- 预算有哪些属性?
- `spent` / `remaining` / `ratio` 说明什么？（说明**预算是"算出来的"**，不是手填的）
- 为什么有 `range: { from, to }`？（说明**预算有周期**，比如月度/年度）

参考答案：这个类型说明预算功能是"**设定额度 + 实时计算已花 + 显示进度**"，而且**支持不同周期。**

步骤 3：做一次影响面分析（本讲核心）

任务：假设要给记录加一个「商家」字段，找出**所有要改的文件。**

```
# ① 契约在哪
Select-String -Path src\shared\types.ts -Pattern "export interface RecordInput" -Context 0

# ② 数据库表结构在哪
Select-String -Path src\main\db\migrations\001_init.ts -Pattern "CREATE TABLE IF NOT EXIST

# ③ 写入 SQL 在哪
Select-String -Path src\main\db\repositories\records.repo.ts -Pattern "INSERT INTO records

# ④ 校验在哪
Select-String -Path src\main\ipc\records.ts -Pattern "function validate"

# ⑤ 界面表单在哪
Select-String -Path src\renderer\src\features\records\RecordForm.tsx -Pattern "const paylo
```

把这五处列出来，就是你的改动清单。 对照 3.1 节那张 8 项的表格，你找到几项?

这个练习的价值：**模块 4 你要真的动手改代码，而这个"影响面分析"是动手前的必备动作。** 不做分析就动手，必然改一半、漏一半。

步骤 4：验证"类型会帮你找漏"

这一步是安全的：你临时加一个字段，看类型检查报什么，然后删掉。**我实测过，加字段后报错 24 条，删掉后回到 0 条。**

1. **先确认现在是干净的**：

```
npm run typecheck
```

预期：没有任何输出（0 错误）。

1. 打开 `src\shared\types.ts`，在 `RecordInput` 里加一行：

```
  merchantTEST: string
```

1. 再跑：

```
npm run typecheck
```

预期看到：**24 条报错**，指向所有"还没处理这个字段"的地方（主要在 `demo.ts`、各种 `*-probe.ts` 里）。

这就是那份"自动生成的待办清单"。

1. **把这行删掉**，再跑一次 `npm run typecheck`，**确认回到 0 错误。**

这一步养成的习惯很重要：**做实验前先确认"基线是干净的"，实验后确认"恢复到干净"。** 否则你会把实验引入的错误当成原本的问题，越查越乱。（模块 4、5 会反复用到这个习惯。）

卡住了怎么办

现象	原因	怎么办			
`typecheck` 报错看不懂	正常，模块 4-4 专讲读报错	只看**文件名和行号**，跳过去看那一行			
加字段后报错太多想放弃	这恰恰说明类型在工作	**这就是它的价值**：否则你要靠记忆找这 8 处			
忘了删测试字段	常见疏忽	跑一次 `npm run typecheck`，干净就说明删对了			
看不懂 `string \	null`	`\	` 是"或"的意思	`string \	null` = "要么是文字，要么是空"

六、自测题

1. 类型在"写代码时"和"运行时"分别起什么作用?

2. 为什么类型定义要放在 `shared\`，而不是主进程和界面各写一份?

3. 给记录加一个字段，为什么要改好几个文件？TypeScript 在这里帮了什么忙？

4. `from?: string` **里的问号是什么意思？**

5. 改动的正确顺序是什么？为什么"先改契约"？

答案与解析

1. 写代码时：类型是护栏，编辑器立刻标红，**防止你写错。运行时**：类型**完全不存在**（构建时被删除，1-3讲实测过），程序不会因为类型错误而报错。 → 推论：**类型防的是开发者，用户输入必须另外校验**（这就是 `validate()` 的作用）。

2. 因为主进程和渲染进程是**两个独立进程**，但必须对数据有**完全一致的理解。** → 各写一份早晚会不一致（比如一边以为金额是元、一边以为是分）。**放** `shared\` **保证只有一份定义。**

3. 因为一个字段会牵涉：类型契约 → 数据库表结构 → 写入 SQL → 读取 SQL → 校验 → 界面表单 → 界面显示。本项目里 `tags` 一个字段就出现在 **14 个文件**中。 → **TypeScript 的作用**：改完契约后，类型检查会**列出所有还没改到的地方**，相当于一份自动生成的待办清单。

4. 表示**这个字段可以不传**（可选字段）。 → 所以 `RecordQuery` 里那些带 `?` 的筛选条件都是可选的——用户可以只用其中一部分。

5. ① 契约（types.ts）→ ② 数据层（迁移 + 仓储）→ ③ 接口层（校验）→ ④ 界面 → ⑤ 跑测试。 → 先改契约是因为**契约是中心**：改它，所有引用它的地方都会报错，**这就是那份自动待办清单。**反过来先改界面，类型检查可能不报错，你就失去了这份清单，必然漏改。

七、术语表

术语	一句话解释	
类型 / Type	对数据形状的要求（是数字还是文字、有哪些字段）	
接口 / Interface	描述一个对象的形状，TypeScript 的主要工具	
契约 / Contract	双方共同承认的数据约定	
类型检查 / Typecheck	在不运行程序的情况下检查类型是否匹配	
编译时 / 运行时	写代码到构建完（类型有效） vs 程序真正跑起来（类型已消失）	
可选字段 / `?`	表示该字段可以不存在	
**联合类型 / `\`	`**	表示"这个值可以是 A 或者 B"
`shared\`	主进程与渲染进程共用的代码目录	
影响面分析	改动前先找出"这次要碰哪些文件"	
待办清单（类型给）	改完契约后，类型检查报错列出的所有待改位置	

八、本讲与项目的对应

- 你查看的文件：`src\shared\types.ts`（505 行 53 个类型）、`src\shared\ipc.ts` 的 `Api`
- 你做的实验：给 `RecordInput` 加字段 → 跑 `typecheck` 看报错 → **记得删掉并确认恢复干净**
- **本讲不修改任何功能代码，不影响你的记账数据**
- 上一讲：3-2 追一条数据流 ｜ 下一讲：**3-4 动手：给列表加一个新列**（模块 3 的实战）

讲义版本：v1 ｜ 对应项目版本：0.1.0